



Module Manifests

Manifest

The manifest file serves to declare a python package as an Odoo module and to specify module metadata.

It is a file called `__manifest__.py` and contains a single Python dictionary, where each key specifies module metadatum.

```
{
    'name': "A Module",
    'version': '1.0',
    'depends': ['base'],
    'author': "Author Name",
    'category': 'Category',
    'description': """
Description text
""",
    # data files always loaded at installation
    'data': [
        'views/mymodule_view.xml',
    ],
    # data files containing optionally loaded demonstration data
    'demo': [
        'demo/demo_data.xml',
    ],
}
```

Available manifest fields are:

name (str , required)

the human-readable name of the module

version (str)

this module's version, should follow [semantic versioning](#) rules

description (str)

website (str)

website URL for the module author

license (str , defaults: LGPL-3)

distribution license for the module. Possible values:

- GPL-2
- GPL-2 or any later version
- GPL-3
- GPL-3 or any later version
- AGPL-3
- LGPL-3
- Other OSI approved licence
- OEEL-1 (Odoo Enterprise Edition License v1.0)
- OPL-1 (Odoo Proprietary License v1.0)
- Other proprietary

category (str , default: Uncategorized)

classification category within Odoo, rough business domain for the module.

Although using [existing categories](#) is recommended, the field is freeform and unknown categories are created on-the-fly. Category hierarchies can be created using the separator / e.g. Foo / Bar will create a category Foo , a category Bar as child category of Foo , and will set Bar as the module's category.

depends (list(str))

Odoo modules which must be loaded before this one, either because this module uses features they create or because it alters resources they define.

When a module is installed, all of its dependencies are installed before it. Likewise dependencies are loaded before a module is loaded.

Note

Module `base` is always installed in any Odoo instance. But you still need to specify it as dependency to make sure your module is updated when `base` is updated.

data (list(str))

List of data files which are only installed or updated in *demonstration mode*

auto_install (bool or list(str) , default: False)

If `True` , this module will automatically be installed if all of its dependencies are installed.

It is generally used for “link modules” implementing synergetic integration between two otherwise independent modules.

For instance `sale_crm` depends on both `sale` and `crm` and is set to `auto_install` . When both `sale` and `crm` are installed, it automatically adds CRM campaigns tracking to sale orders without either `sale` or `crm` being aware of one another.

If it is a list, it must contain a subset of the dependencies. This module will automatically be installed as soon as all the dependencies in the subset are installed. The remaining dependencies will be automatically installed as well. If the list is empty, this module will always be automatically installed regardless of its dependencies and these will be installed as well.

external_dependencies (dict(key=list(str)))

A dictionary containing python and/or binary dependencies.

For python dependencies, the `python` key must be defined for this dictionary and a list of python modules to be imported should be assigned to it.

For binary dependencies, the `bin` key must be defined for this dictionary and a list of binary executable names should be assigned to it.

The module won't be installed if either the python module is not installed in the host machine or the binary executable is not found within the host machine's `PATH` environment variable.

application (bool , default: False)

Whether the module should be considered as a fully-fledged application (`True`) or is just a technical module (`False`) that provides some extra functionality to an existing application module.

assets (dict)

A definition of how all static files are loaded in various assets bundles. See the [assets](#) page for more details on how to describe bundles.

installable (bool default: True)

the author is the maintainer.

`{pre_init, post_init, uninstall}_hook ( str )`

Hooks for module installation/uninstallation, their value should be a string representing the name of a function defined inside the module's `__init__.py`.

`pre_init_hook` takes an `env` as its only argument, this function is executed prior to the module's installation.

`post_init_hook` takes an `env` as its only argument, this function is executed right after the module's installation.

`uninstall_hook` takes an `env` as its only argument, this function is executed after the module's uninstallation.

These hooks should only be used when setup/cleanup required for this module is either extremely difficult or impossible through the api.

`active ( bool )`

Deprecated. Replaced by `auto_install`.

Get Help

[Contact Support](#)[Ask the Odoo Community](#)